

Design and implementation of a probabilistic bird classification system based on selected modality and meta-information about the distribution of birds in geographical space

Julianna Godziszewska

Wrocław University of Science and Technology

Keywords: Machine Learning · Data Science · Biodiversity.

Abstract

The aim of this work was to develop and evaluate a system of recognizing bird species based on recordings of their singing, with a focus on geographical location and its impact on classification. The study shows whether including spatial metadata enhances species recognition accuracy. To achieve this, two classification models were developed. One model is based on deep neural networks, which is responsible for species classification using spectrogram representations of bird sounds. Another one uses kernel density estimation to perform classification based on species occurrence coordinates and to model their geographical distribution at the same time. These models were integrated into a multi-classifier system to take advantage of their complementary strengths.

System architecture

The system combines both acoustic and spatial information to improve classification performance. It consists of two models:

- Model A is designed to classify audio data. It is implemented as a deep neural network consisting of convolutional layers.
- Model B is responsible for classifying geographical coordinates. It consists of kernel density estimators built separately for each class. For each class, a probability density function is estimated based on the training data. When a new sample is evaluated, each estimator calculates its density. The sample is then assigned to the class with the highest estimated density.

These two models are combined using an ensemble architecture. The final decision is made by a weighted average ensemble rule, where each expert, that is Model A and Model B contributes with a different weight.

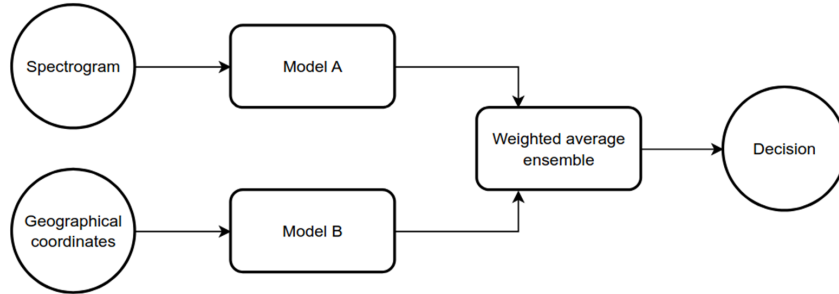


Fig. 1. System architecture

Data acquisition

All the data that was used to build the bird recognition system comes from the Global Biodiversity Information Facility, which is an international and open-source network funded by the world’s governments. It provides access to data about all types of life on Earth. Downloaded data comes from six publicly available datasets and includes a total of 24 934 records:

- Xeno-canto - Wildlife sounds from around the world [9]
- iNaturalist Research - grade Observations [3]
- Animal Sound Archive [1]
- Observation.org, Nature data from around the World [5]
- My naturesounds - nature observations with sound recordings [7]
- BoBO - Botanic Garden and Botanical Museum Berlin Observations [2]

All collected data consists of audio recordings, geographical coordinates and the bird species name. The dataset was filtered to include only bird species found in Poland, based on the geographical coordinates of the recordings.

Dataset

In order to create the dataset for the system, the raw audio recordings were converted into spectrograms, which represent a signal both in time and frequency domains. The audio signal was sampled at a native sampling rate and then divided into time frames of size 2048 samples, with a hop size of 1024 samples. Each frame was smoothed using a Hanning window function to reduce spectral leakage during the frequency analysis. A Fast Fourier Transform was applied

⁰ <https://gbif.com>

to convert each frame into the frequency domain. The resulting frequency representation was squared to obtain the power spectral density, which was then converted into a logarithmic scale to form the spectrogram¹.

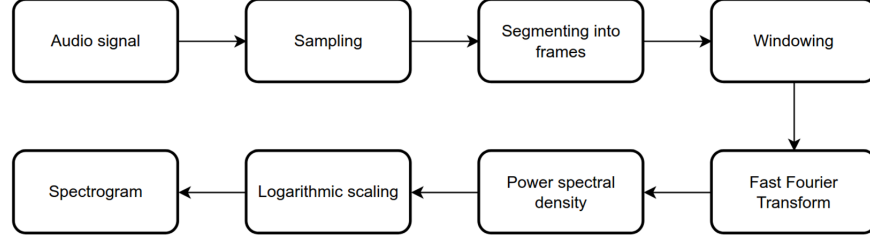


Fig. 2. Process of generating a spectrogram

Due to the long duration of some recordings, it was necessary to divide the spectrograms into smaller segments in order to reduce the number of attributes. I decided to split the spectrograms every five frames and then flatten each segment into a one-dimensional vector. Each such spectrogram fragment is treated as a separate record, independent of other fragments from the same recording.

The dataset was split into three parts using stratified sampling to preserve the original class distribution across all subsets. 70% of the data was used for training, 15% for validation, and 15% for testing.

The chart below (Fig. 3) presents the a priori class probability distribution, based on dataset occupancy for each class. The dataset consists of 306 classes, and as shown, the class distribution is heavily imbalanced. We can observe that a small number of classes dominate the dataset, while most classes have very low representation. The classes are sorted in descending order based on their dataset occupancy, so the most frequent classes appear on the left side of the plot.

¹ [Spectrogram code reference](#)

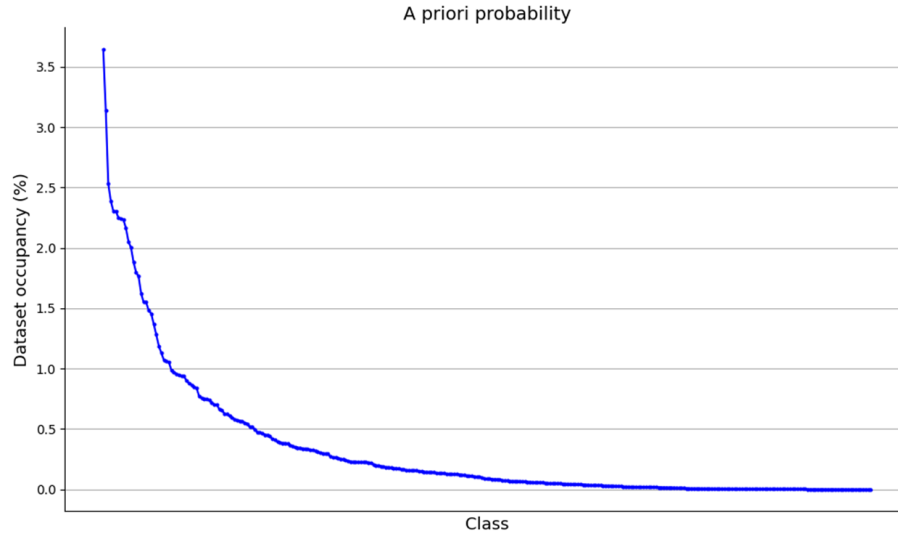


Fig. 3. Dataset occupancy for each class

Model A

Model A focuses on the classification of audio recordings. It is built as a deep neural network that consists of four convolutional layers grouped with max-pooling operations, followed by fully connected layers².

The model uses ReLU as the activation function and applies log-softmax at the output to produce class probabilities.

Due to the large size of the dataset, the training time was significant — each epoch took over 8 hours on average on available the equipment. Therefore, the model was trained for only 8 epochs.

² [CNN code reference](#)

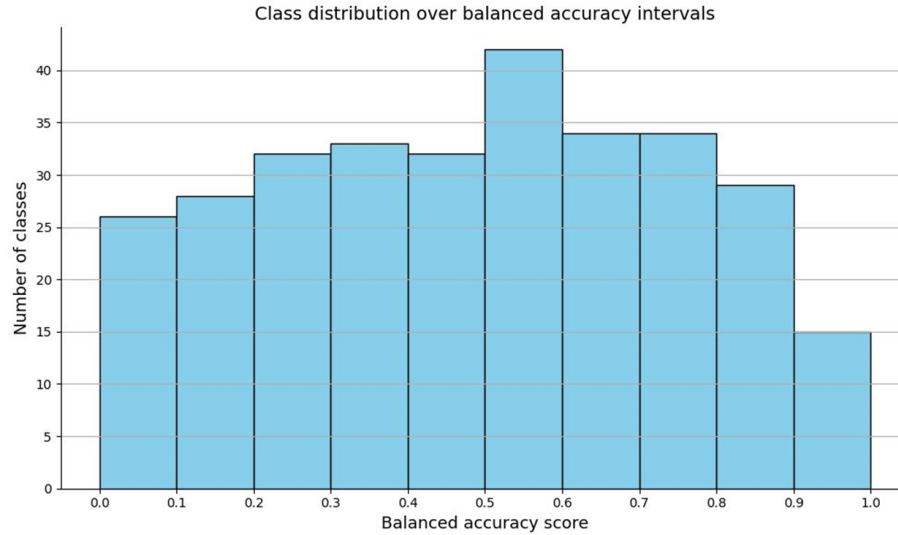


Fig. 4. Class distribution over balanced accuracy scores of model A

The histogram above (Fig. 4) shows the distribution of classes across intervals of the balanced accuracy score. The distribution appears relatively even. The least frequent result is in the 90% to 100% range, where only 15 classes reached this level of performance.

Model B

Model B is designed to classify samples based on their geographical coordinates. It relies on kernel density estimation, where a separate density model is fitted for each class. The classification is performed by selecting the class with the highest estimated density for a given input location.

```
from sklearn.neighbors import KernelDensity as KD

def create_estimators(X_train, y_train, kernel):

    kde_estimators = {}
    unique_classes = np.unique(y_train)

    for class_label in unique_classes:
        h = silverman_bandwidth(X_train[class_indices])
        kde = KD(kernel=kernel, bandwidth=h).fit(X_train[class_indices])
        kde_estimators[class_label] = kde
```

```
return kde_estimators
```

Listing 1.1. Implementation of kernel density estimators

```
import numpy as np

def predict_classes(kde_estimators, X_test, y_test):

    predictions = []
    test_classes = np.unique(y_test)

    for sample in X_test:
        densities = {}

        for class_label, kde in kde_estimators.items():
            if class_label not in test_classes:
                continue

            log_density = kde.score_samples(sample.reshape(1, -1))
            densities[class_label] = np.exp(log_density)

        predicted_class = max(densities, key=densities.get)
        predictions.append(predicted_class)

return np.array(predictions)
```

Listing 1.2. Implementation of classification on test data

During the estimation of kernel density functions, it is very important to choose an appropriate kernel bandwidth parameter, which controls the level of smoothing applied to the distribution. I wanted to select a method that would best match my data. Silverman’s rule is a widely used method which is designed for unimodal, symmetric, and normally distributed data.

In my case, the input data consisted of raw geographical coordinates, which were not standardized or scaled. Therefore, I decided to apply a modified version of Silverman’s rule, as it offers better robustness in the presence of outliers and can capture multimodal characteristics of the data more effectively.

That version of Silverman’s rule takes into account an additional measure of data dispersion — the interquartile range. This measure is more resistant to outliers than the standard deviation. Additionally, this method includes a reduced coefficient of 0.9 instead of 1.06, which improves the identification of multimodal features. The modified version of Silverman’s rule is described by the following formula [8]:

$$h = 0.9 \cdot \min\left(\sigma, \frac{IQR}{1.35}\right) n^{-\frac{1}{5}}$$

where:

h - kernel bandwidth,

σ - standard deviation of the sample,

IQR - interquartile range, i.e., the difference between the third quartile and the first quartile,

n - number of samples in the dataset.

To illustrate the impact of kernel selection and bandwidth adjustment, a series of experiments were conducted. The first set of experiments compared all available kernel types in Scikit-learn[6] library using the bandwidth parameter determined by modified Silverman's method.

Table 1. Balanced accuracy scores for the bandwidth parameter h determined using Silverman's method for different kernel types

Kernel Type	Balanced Accuracy Score
Gaussian (Gauss)	0.713 %
Tophat (Rectangular)	0.369 %
Epanechnikov	1.385 %
Exponential	1.112 %
Linear	1.598 %
Cosine	1.471 %

The table (Table 1) shows balanced accuracy scores for all kernel types. The best result achieved linear kernel, reaching approximately 1.6 % of accuracy. We can observe that all scores for different kernels are relatively low. They are not satisfying, therefore I started exploring how they could be improved. I decided to experiment with reducing the bandwidth values by several orders of magnitude, but still using the modified Silverman's rule.

Therefore, the same kernel types were tested with a reduced kernel bandwidth, decreased by two orders of magnitude.

Table 2. Balanced accuracy scores for the bandwidth parameter h determined using Silverman's method, decreased by two orders of magnitude for different kernel types

Kernel Type	Balanced Accuracy Score
Gaussian (Gauss)	14.076 %
Tophat (Rectangular)	14.238 %
Epanechnikov	15.547 %
Exponential	14.970 %
Linear	15.938 %
Cosine	15.603 %

The results from the table above (Table 2) shows balanced accuracy scores for all kernel types after decreasing kernel bandwidth by two orders of magnitude. All kernel types achieved noticeably higher balanced accuracy scores compared to the previous configuration. Again, the linear kernel achieved the best result, this time reaching approximately 16% balanced accuracy.

Model C

Model C is a mixture of experts that combines the predictions of two individual models: Model A and Model B. The final prediction is obtained as a weighted average of the outputs from these experts, where the weights can be adjusted to control each expert's influence.

The heatmap below (Fig. 5) illustrates the performance of the mixture of experts in terms of balanced accuracy, depending on two key parameters: the kernel bandwidth used in Model B and the weight assigned to expert A. I assumed that Model A acts as the dominant expert, so I considered expert A weights of 0.6, 0.7, 0.8, 0.9, and 1.

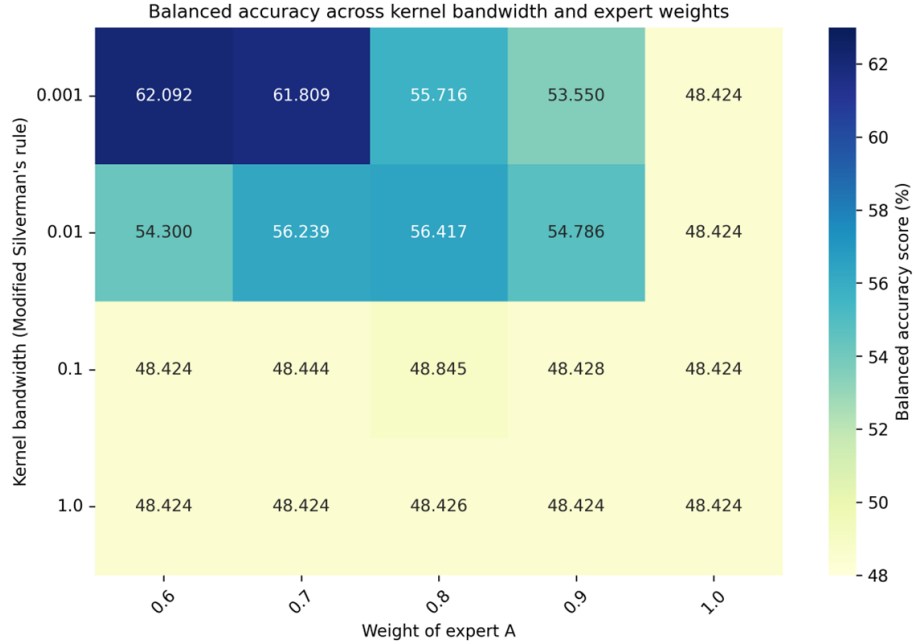


Fig. 5. Impact of Kernel Bandwidth and Expert Weights on Balanced Accuracy

For the original modified Silverman’s rule, as well as for the version scaled down by only one order of magnitude, the improvement in accuracy was minimal. The highest balanced accuracy score was achieved with a kernel bandwidth reduced by three orders of magnitude and an expert A weight of 0.6, reaching approximately 62.1%.

However, despite this high score, I decided to reject this configuration due to the extremely low kernel bandwidth. I was concerned that the model might be overfitting to the training data. I decided to chose the smallest bandwidth value that enhanced the classification which is kernel bandwidth decreased by two orders of magnitude. The final configuration of Model C used in the system was as follows:

- Kernel bandwidth: modified Silverman’s rule, scaled down by two orders of magnitude,
- Kernel type: linear,
- Expert A weight: 0.8,
- Expert B weight: 0.2.

The histogram below (Fig. 6) shows the distribution of the number of classes for each interval of the balanced accuracy score after applying Model C.

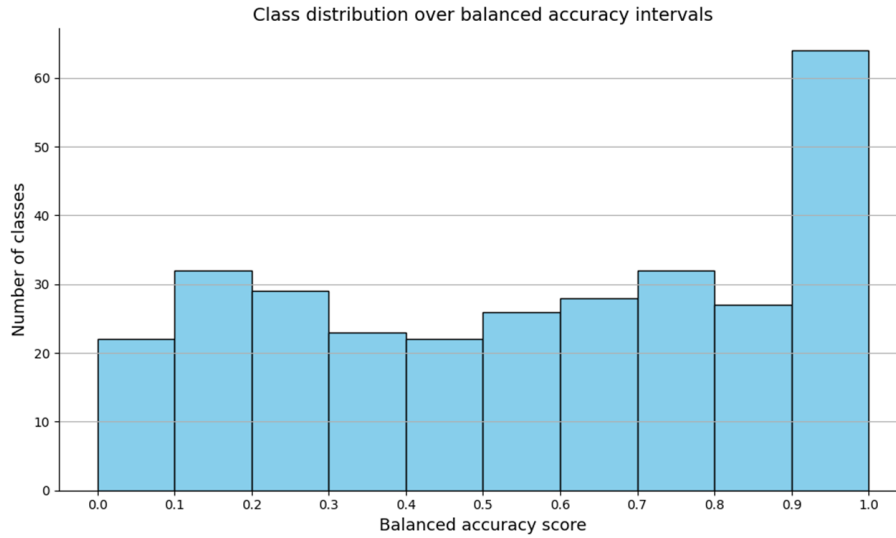


Fig. 6. Class distribution over balanced accuracy scores intervals of model C

Compared to the results for Model A, the number of classes that fall into the highest accuracy interval increased approximately four times.

Conclusions

The main goal of this project was to develop a bird song recognition system using both audio recordings and geographical metadata. The system integrates two models into the mixture of experts that is able to classify spectrograms and location data.

Choosing the appropriate kernel bandwidth is crucial for obtaining a well-balanced density estimation. If parameter h is too small, the model captures too many local variations, leading to overfitting and reduced generalization. On the other hand, an excessively large kernel bandwidth over-smooths the density function, making it difficult to distinguish between close data points. In practice, the kernel bandwidth can be interpreted as the "resolution" of the data: a smaller h provides a high-resolution view with fine details, while a larger h results in a low-resolution, more generalized representation of the distribution [4].

This effect was clearly observed in the performance of Model C. The results showed that reducing the kernel bandwidth parameter by two orders of magnitude led to a significant increase in classification performance, improving the balanced accuracy of the final system to approximately 56.4%. Furthermore, the number of bird species recognized with near-perfect accuracy increased approximately four times compared to Model A alone.

References

1. Museum für Naturkunde Berlin. Animal sound archive. Occurrence dataset, 2024. Dostęp poprzez GBIF.org 23 listopada 2024. [doi:10.15468/0bpalr](https://doi.org/10.15468/0bpalr).
2. Botanic Garden and Botanical Museum Berlin. Bobo - botanic garden and botanical museum berlin observations. Occurrence dataset, 2019. Dostęp poprzez GBIF.org 23 listopada 2024. [doi:10.15468/9112gz](https://doi.org/10.15468/9112gz).
3. iNaturalist contributors. inaturalist research-grade observations. Occurrence dataset, 2024. Dostęp poprzez GBIF.org 23 listopada 2024. [doi:10.15468/ab3s5x](https://doi.org/10.15468/ab3s5x).
4. Mateusz Kobos. Combination of independent kernel density estimators in classification. In *2009 International Multiconference on Computer Science and Information Technology*, pages 57–63, 2009. [doi:10.1109/IMCSIT.2009.5352749](https://doi.org/10.1109/IMCSIT.2009.5352749).
5. Observation.org. Observation.org, nature data from around the world. Occurrence dataset, 2024. Dostęp poprzez GBIF.org 23 listopada 2024. [doi:10.15468/5nilie](https://doi.org/10.15468/5nilie).
6. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
7. V. Runnel. My naturesounds - nature observations with sound recordings. PlutoF. Occurrence dataset, 2024. Dostęp poprzez GBIF.org 23 listopada 2024. [doi:10.15468/hnheni](https://doi.org/10.15468/hnheni).
8. Simon Sheather. Density estimation. *Statistical Science*, 19, 11 2004. [doi:10.1214/088342304000000297](https://doi.org/10.1214/088342304000000297).
9. W. Vellinga. Xeno-canto - bird sounds from around the world. Occurrence dataset, 2024. Dostęp poprzez GBIF.org 23 listopada 2024. [doi:10.15468/qv0ksn](https://doi.org/10.15468/qv0ksn).